



Погружение уточняющих типов данных в системы с зависимыми типами



Соколов Павел Павлович

ФКН НИУ ВШЭ, Москва

Основные понятия

В теории типов основным объектом изучения являются **системы типов** — формальные математические модели языков программирования (либо чисто логических исчислений), состоящие из описания грамматических форм языка, правил вычисления, и, наконец, **правил типизации**, классифицирующих выражения по тому, какие “значения” они вычисляют (понятие значения, конечно, крайне условно и зависит от языка). Особый интерес представляют системы типов, позволяющие одновременно кодировать и алгоритмы, и доказательство их корректности. Здесь помогает принятие парадигмы **Утверждения есть (некоторые) типы**, отождествляющее понятия из левой колонки в таблице ниже с понятиями из правой колонки:

Таблица 1: Парадигма Propositions as (Some) Types

Логика	Теория типов
Утверждение P	Тип P
Теорема T	Населённый тип T
Предположение H	Переменная $x : H$
Доказательство $\Gamma \vdash T$	Программа $\Gamma \vdash \pi : T$
Устранение сечений	Нормализация вычислений
Непротиворечивость	$\nexists t. \vdash t : \perp$
$\vdash P \wedge Q$	$\vdash (p : P, q : Q) : P \times Q$
$\vdash P \rightarrow Q$	$\vdash (\lambda(x : P).q : Q) : P \rightarrow Q$
Кванторы	Зависимые типы
$\vdash \exists x \in X. P(x)$	$\vdash (w : X, \pi : P(w)) : \Sigma_{x:X} P(x)$
$\vdash \forall x \in X. P(x)$	$\vdash (\lambda(x : X). \pi : P(x)) : \Pi_{x:X} P(x)$

В рамках этой парадигмы поиск доказательства сводится к синтезу программы подходящего типа; корректность доказательства проверит компилятор. Правда, выписывать такое доказательство трудно и долго, так что желательно максимально автоматизировать и упростить этот процесс.

Историческая справка

Первой успешной практической реализацией языка на основе парадигмы Propositions as Types следует считать систему Rocq (ранее Coq) Тьерри Коканда, основанную на Исчислении Конструкций [1]. В этой системе постулируется существование двух сортов: **Prop** (сорт утверждений) и **Type** (сорт типов данных), типов (зависимых) функций, связывающих сорта, а также индуктивных определений (с их помощью можно как постулировать новые утверждения, так и строить пользовательские типы данных). Другие аналогичные системы [2, 3] основываны на той же самой базовой идее, но проблема автоматизации доказательств решается немного по-разному.

- В Rocq [4] автоматизация достигается посредством **тактик**, специальных команд, генерирующих собственно код доказательств;
- В Lean [3] тоже используют тактики, а также охотно пользуются аксиомами классической логики и теории множеств, что приближает доказательства в Lean к обычной, “бумажной” математике;
- В Agda [2] тактик нет; вместо этого эксплуатируется рефлексия и продвинутый алгоритм **элаборации**, позволяющий программисту опускать большинство технических аннотаций типов.

В частности, автоматизация в Agda больше сконцентрирована на реализации **солверов** — алгоритмов, генерирующих доказательство в рамках некоторой ограниченной теории (ведь для всей системы типов целиком поиск доказательства — неразрешимая задача, как и для классической логики первого порядка). С наглядным примером реализации этого подхода можно ознакомиться в [5].

Альтернативный подход — изначально ограничить систему типов таким образом, чтобы в ней были выразимы только разрешимые предикаты; кроме этого, желательно объединить программу и доказываемые про неё свойства в единую сущность: $\{x : T \mid \varphi(x)\}$. Такие типы называются **уточняющими**; гибкость по сравнению с обычными зависимыми типами достигается посредством следующего правила [6]:

$$\frac{\Gamma \vdash t : \{x : T \mid \varphi(x)\} \quad \llbracket \Gamma, x : T \rrbracket \models \varphi(x) \Rightarrow \psi(x)}{\Gamma \vdash t : \{x : T \mid \psi(x)\}}$$

Обязательство по доказательству $\varphi \Rightarrow \psi$ возлагается на компилятор; в статье, на которую мы ссылаемся выше, предлагается использовать любой готовый SMT-солвер. К сожалению, такое решение не расширяемо: набор автоматически доказуемых утверждений ограничен возможностями используемого солвера. Кроме этого, полученная система не самодостаточна.

Основные результаты

Мы предлагаем систему с зависимыми типами, поддерживающую уточняющие типы данных на уровне библиотеки, где множество автоматически доказуемых утверждений расширяемо пользователем. При этом система не опирается на внешний источник доказательств, а строит их самостоятельно. Это достигается с помощью алгоритма **разрешения ограничений** [7], используемого в компиляторах функциональных языков программирования для поддержки перегрузки операторов. Из интересных деталей, по сравнению с системой, кратко описанной в [6], мы добавляем следующее правило:

$$\frac{\Gamma \vdash t : \{x : T \mid \varphi(x)\} \quad \Gamma \vdash u : \{y : U(t : T) \mid \psi(t, y)\}}{\Gamma \vdash (t, u) : \{(x, y) : \Sigma_{x:T} U(x) \mid \varphi(x) \wedge \psi(x, y)\}}$$

Благодаря ему, система получает возможность рассуждать о совокупностях объектов (например, об алгебраических структурах и операциях на них) эргономичным для пользователя образом.

Идеи доказательств

Доказательство сохранения типизации производится стандартным образом. Логическая непротиворечивость следует из сводимости нашей системы к Исчислению конструкций [1], про которое была доказана [8] взаимная консистентность с ZFC (+ необходимое количество недостижимых кардиналов). Ключевой деталью в построении сведения является разрешимость процедуры поиска доказательств, благодаря которой сведение получается эффективным.

Выводы и дальнейшие направления исследований

Таким образом, мы получили исчисление, в рамках которого автоматизируется поиск тривиальных заключений из доказанных пользователем теорем. Интересно также рассмотреть следующие расширения:

- Практическую реализацию системы;
- Интегрирование полноценных солверов разрешимых теорий внутрь системы;
- Взаимодействие уточняющих типов с гомотопическими теориями типов;
- Оптимизация сложности генерируемых доказательств;
- Взаимодействие уточняющих типов с эффективной компиляцией.

Список литературы

- [1] T. Coquand and G. Huet, “The calculus of constructions,” *Information and Computation*, vol. 76, no. 2, pp. 95–120, 1988.
- [2] U. Norell, Towards a practical programming language based on dependent type theory. PhD thesis, Department of Computer Science and Engineering, Chalmers University of Technology, SE-412 96 Göteborg, Sweden, September 2007.
- [3] L. d. Moura and S. Ullrich, “The Lean 4 theorem prover and programming language,” in Automated Deduction – CADE 28 (A. Platzer and G. Sutcliffe, eds.), (Cham), pp. 625–635, Springer International Publishing, 2021.
- [4] T. C. D. Team, “The Coq proof assistant,” Sept. 2024.
- [5] P. van der Walt and W. Swierstra, “Engineering Proof by Reflection in Agda,” in Lecture Notes in Computer Science (R. Hinze, ed.), vol. 8241 of Lecture Notes in Computer Science, (Oxford, United Kingdom), pp. 157–173, Springer, Aug. 2012.
- [6] G. Shen and L. Kuper, “Toward SMT-based refinement types in Agda,” 2021.
- [7] D. Selsam, S. Ullrich, and L. de Moura, “Tabled typeclass resolution,” 2020.
- [8] B. Werner, “Sets in types, types in sets,” in Theoretical Aspects of Computer Software (M. Abadi and T. Ito, eds.), (Berlin, Heidelberg), pp. 530–546, Springer Berlin Heidelberg, 1997.